Київський національний університет імені Тараса Шевченка

Факультет інформаційних технологій

Кафедра програмних систем і технологій

Звіт

з лабораторної роботи № 3

з дисципліни “Об’єктно-орієнтоване програмування”

Тема: «Створення проекту і файлу локальної бази даних. Основи LINQ»

«Автоматизована система “Склад запчастин для авто”»

Виконав: студент гр. ІПЗ-21, підгрупа 1

Агапов Олександр Олексійович

Перевірила: доц. Зубик Л. В.

Київ — 2026

1. МЕТА ТА ЗАВДАННЯ РОБОТИ

Тема лабораторної роботи — «Створення проекту і файлу локальної бази даних. Основи LINQ». Мета — створити окрему базу даних для системи «Склад запчастин для авто» та навчитися отримувати з неї потрібні відомості за допомогою LINQ.

Для цього використано C#, ASP.NET Core MVC, Entity Framework Core, SQLite і LINQ. Потрібно зв’язати таблиці, заповнити їх демонстраційними даними, показати не менше п’яти різних запитів, фільтрацію, підсумок за вибраний період і операції додавання, редагування та видалення.

2. СТРУКТУРА БАЗИ ДАНИХ

У Lab 3 створено окрему базу AutoPartsWarehouseLab3.db. Основні сутності: AutoPart (запчастина), Supplier (постачальник), PurchaseOrder (замовлення), PurchaseOrderItem (позиція замовлення), VehicleModel (модель автомобіля) і PartIssue (видача запчастини). У кожної основної сутності є первинний ключ Id.

PurchaseOrder має зовнішній ключ SupplierId. Позиція замовлення пов’язана із замовленням через PurchaseOrderId, а із запчастиною — через AutoPartId; окремий клас потрібен, бо позиція зберігає кількість. PartIssue також посилається на AutoPart. Сумісність запчастин і моделей авто має зв’язок багато-до-багатьох. EF Core створює для нього фізичну таблицю AutoPartVehicleModel з двома зовнішніми ключами та складеним первинним ключем.

Отже, у прикладній моделі шість основних таблиць, а у фізичній базі — сім таблиць предметної області разом із проміжною. Окремо EF Core веде службову __EFMigrationsHistory; sqlite_sequence — службова таблиця SQLite. Це перевірено на базі, створеній з поточних міграцій.

3. СТВОРЕННЯ ТА ПОЧАТКОВЕ ЗАПОВНЕННЯ БАЗИ

У Program.cs налаштовано WarehouseContext для SQLite. Під час запуску застосовуються міграції, після чого викликається SeedData.Initialize. Міграції послідовно створюють таблиці, ключі та індекс унікальності для пари назва запчастини — виробник.

```
context.Database.Migrate();
SeedData.Initialize(context);
```

SeedData перевіряє, чи порожня відповідна таблиця, і лише тоді додає приклади. На чистій тестовій базі отримано 10 запчастин, 3 постачальники, 2

замовлення, 3 позиції замовлень, 2 моделі авто, 4 зв’язки сумісності та 6 видач. Повторний запуск не створює повторних наборів записів.

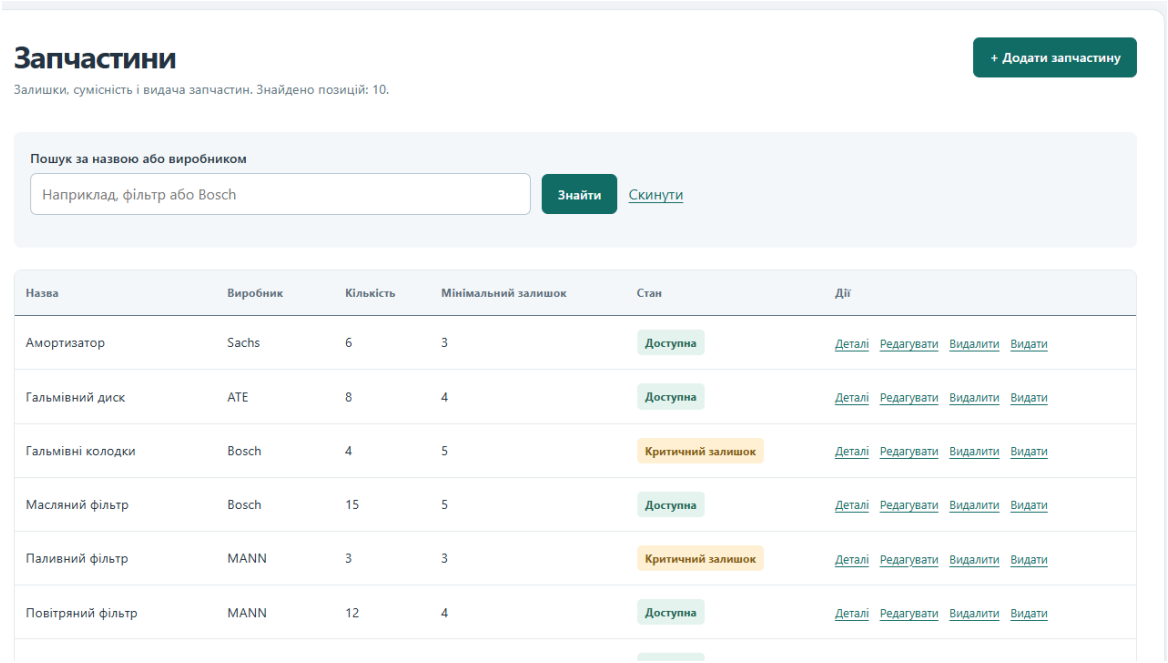


Рисунок 1 — Список запчастин після початкового заповнення

4. ЗАПИТИ LINQ І РЕЗУЛЬТАТИ

Запити виконуються в ReportsController. Контролер формує умови для DbSet, а ToListAsync або SumAsync виконує їх у БД. На знімках показано дані, отримані з окремої тестової копії застосунку 27.10.2026.

4.1. Пошук за виробником

```
parts = parts.Where(part =>
```

part.Manufacturer.Contains(manufacturer));

```
var result = await parts.OrderBy(part => part.Name).ToListAsync();
```

Where залишає запчастини вибраного виробника. Для Bosch відображено три позиції: гальмівні колодки, масляний фільтр і щітки склоочисника. Сортування за назвою виконується перед отриманням списку.

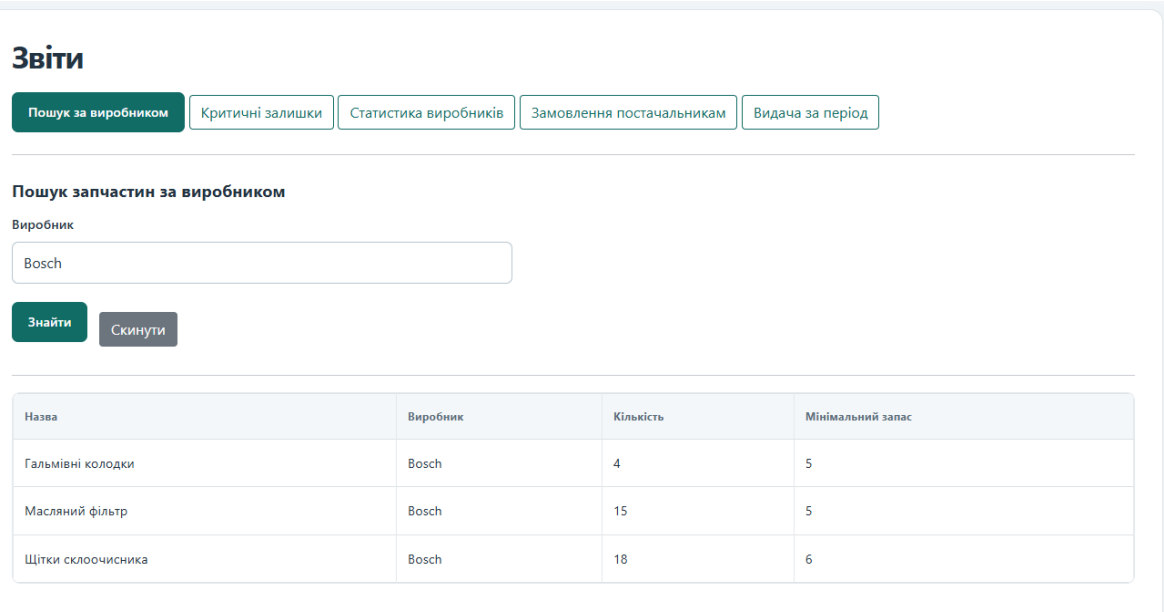


Рисунок 2 — Фільтрація запчастин за виробником Bosch

4.2. Критичні залишки

```
var parts = await _context.AutoParts
```

.Where(part => part.QuantityInStock > 0 &&

part.QuantityInStock <= part.MinimumStock)

.OrderBy(part => part.QuantityInStock).ToListAsync();

Запит показує запчастини, кількість яких додатна, але не перевищує мінімальний запас. У тестовій базі це ремінь ГРМ (2 із мінімуму 4), паливний фільтр (3 із 3) та гальмівні колодки (4 із 5).

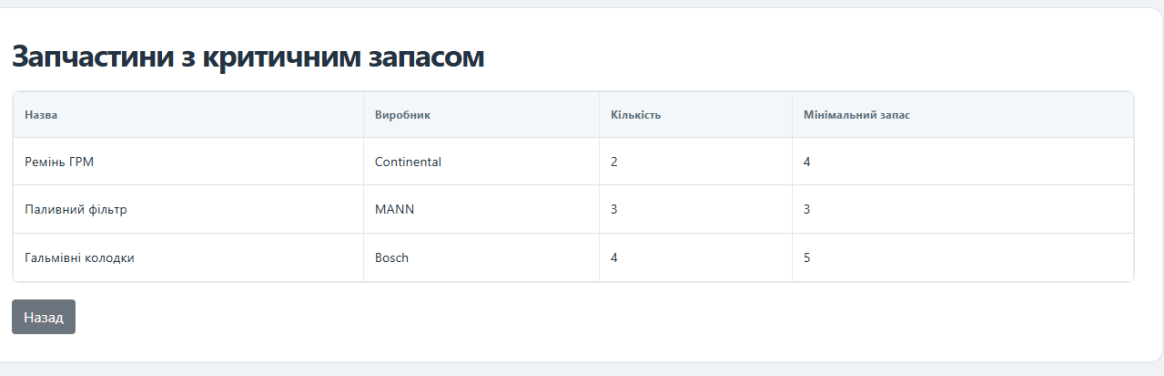


Рисунок 3 — Запчастини з критичним залишком

4.3. Статистика за виробниками

```
_context.AutoParts.GroupBy(part => part.Manufacturer)
```

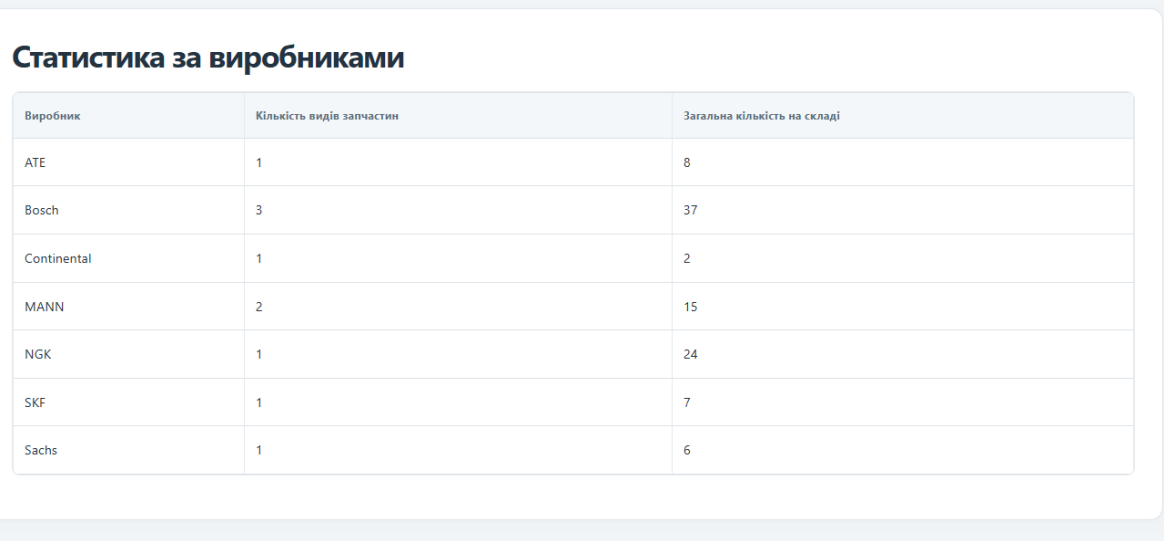
.Select(group => new ManufacturerStockViewModel {

Manufacturer = group.Key, PartCount = group.Count(),

TotalQuantity = group.Sum(part => part.QuantityInStock)

}).OrderBy(item => item.Manufacturer).ToListAsync();

GroupBy об’єднує позиції за виробником; group.Key — його назва. Count рахує види запчастин, Sum — загальну кількість на складі. Наприклад, для Bosch результат — 3 види й 37 одиниць.



Статистика за виробниками

Виробник	Кількість видів запчастин	Загальна кількість на складі
ATE	1	8
Bosch	3	37
Continental	1	2
MANN	2	15
NGK	1	24
SKF	1	7
Sachs	1	6

Рисунок 4 — Кількість видів і залишок за виробниками

4.4. Замовлення постачальникам

```
_context.PurchaseOrders.Include(order => order.Supplier)
```

.Include(order => order.Items)

.ThenInclude(item => item.AutoPart)

.OrderByDescending(order => order.OrderDate).ToListAsync();

Include і ThenInclude завантажують постачальника, позиції замовлення та їхні запчастини. У тестовій базі відображено два замовлення: EuroParts із ременем ГРМ та AutoTech Supply із масляним фільтром і гальмівними колодками.

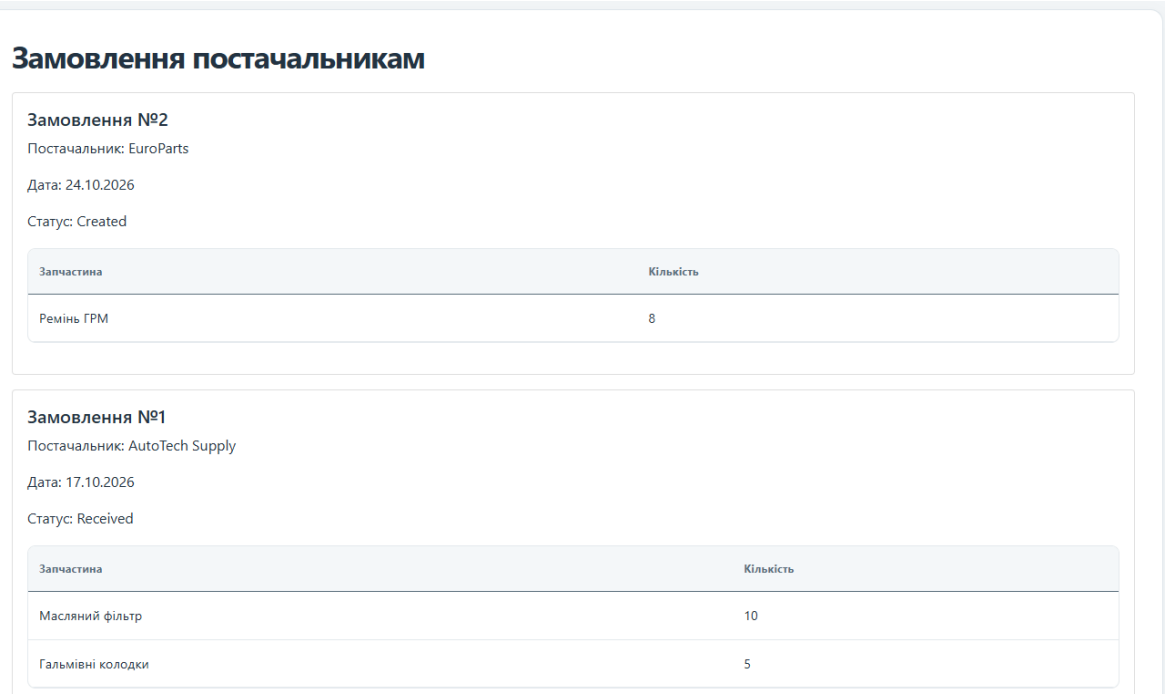


Рисунок 5 — Замовлення разом із постачальниками та позиціями

4.5. Видачі за вибраний період

```
if (from.HasValue)
    issues = issues.Where(issue => issue.IssuedAt >= from.Value);

if (to.HasValue)
    issues = issues.Where(issue =>
        issue.IssuedAt < to.Value.Date.AddDays(1));

var summary = await issues
    .GroupBy(issue => new
    {
        issue.AutoPartId,
        issue.AutoPart.Name,
        issue.AutoPart.Manufacturer
    })
    .Select(group => new IssuedPartSummary
    {
        PartName = group.Key.Name,
        Manufacturer = group.Key.Manufacturer,
        TotalIssued = group.Sum(issue => issue.Quantity)
    })
    .OrderBy(item => item.PartName)
    .ToListAsync();
```

Спочатку запити про видачу фільтруються за обраним періодом. Далі GroupBy об'єднує записи видач однієї конкретної запчастини: AutoPartId є її ідентифікатором, а назва та виробник потрібні для відображення у підсумковій таблиці. Sum обчислює кількість одиниць лише всередині відповідної групи, тому кількості різних товарів не змішуються.

Select формує об'єкт IssuedPartSummary для кожної запчастини. IssuedByPeriodViewModel передає до View список підсумків Summary і список

окремих операцій Issues, тому на сторінці одночасно відображаються агреговані дані за запчастинами та детальна історія видач.

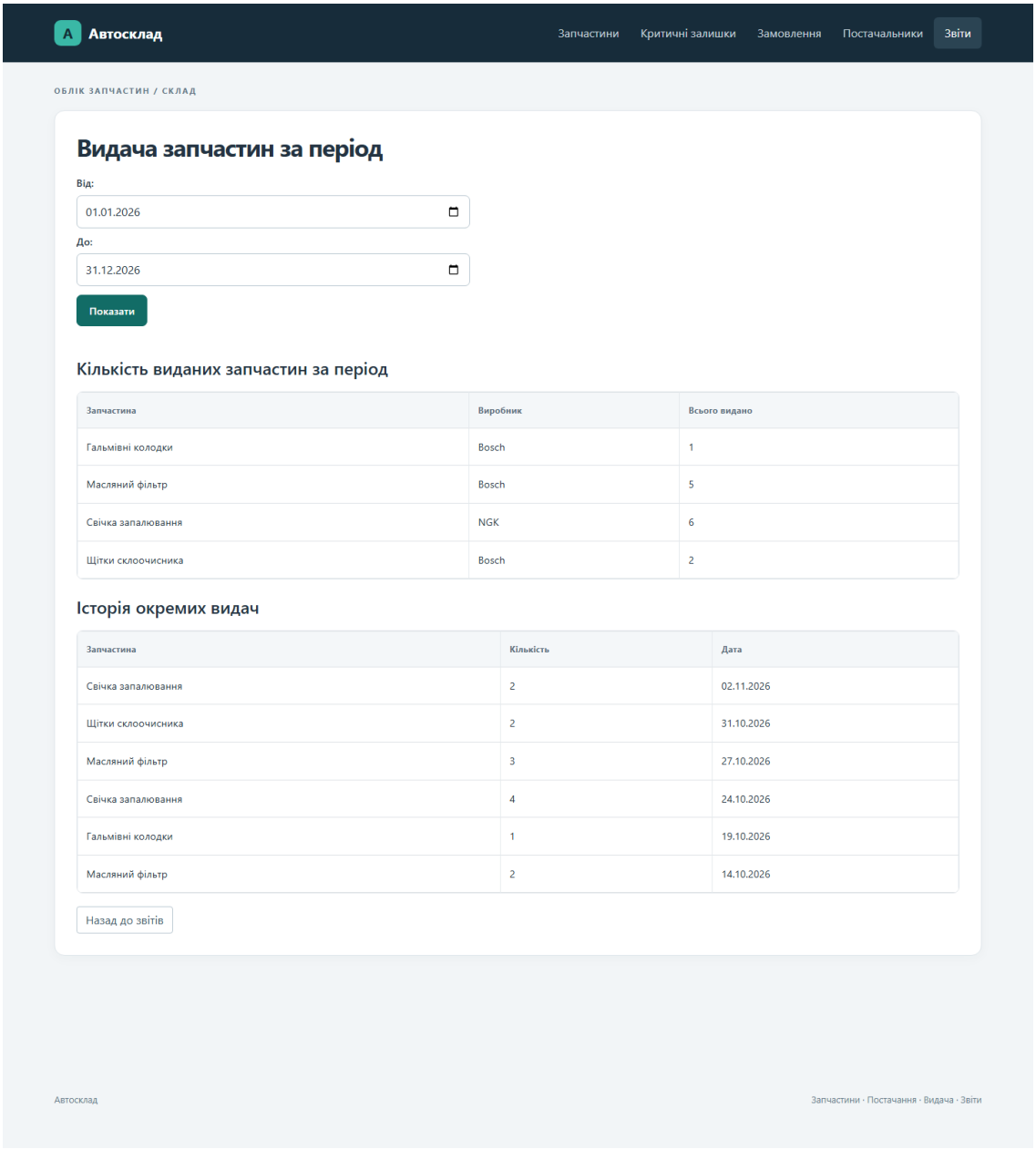


Рисунок 6 — Підсумки видач за запчастинами та історія операцій за вибраний період

5. ДОДАВАННЯ, РЕДАГУВАННЯ ТА ВИДАЛЕННЯ

Операції виконуються у AutoPartsController. GET-методи відкривають форми, POST-методи перевіряють ModelState, зберігають зміни через SaveChangesAsync і повертають користувача до списку. Створення перевіряє унікальність назви разом із виробником; редагування оновлює знайдений запис; видалення вимагає підтвердження й заборонене, якщо запчастина вже має історію замовлень або видач.

На окремій тестовій БД додано демонстраційну запчастину з кількістю 11, змінено кількість на 12, після чого запис видалено. Основна база Lab 3 при перевірці не змінювалася.

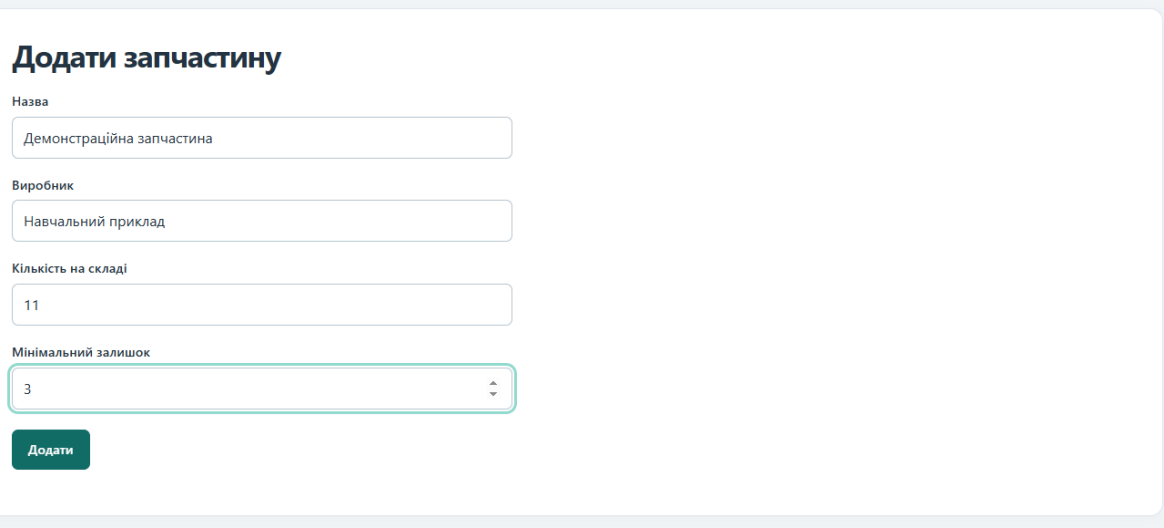


Рисунок 7 — Форма створення демонстраційної запчастини

6. ВИСНОВОК

У цій лабораторній стало видно, навіщо складу потрібна окрема база даних: запчастини, замовлення й видачі зберігаються не розрізнено, а пов’язані між собою. LINQ допоміг отримати з цих записів конкретні відповіді — які запчастини закінчуються, скільки їх є за кожним виробником і скільки було видано за обрані дати. Групування дає змогу побачити підсумки, не переглядаючи всі записи по одному. Створення, редагування й видалення запчастини перевірено на окремій тестовій базі, щоб не змінювати основні дані. Кількість виданих запчастин у звіті за період агрегується окремо для кожної запчастини, тому підсумки різних товарів не об'єднуються в одне загальне число.

ВИКОРИСТАНІ ДЖЕРЕЛА

1. Матеріали лекцій з LINQ та оформлення звітів, надані викладачем. 2. Вихідний код застосунку AutoPartsWarehouseLab3, лабораторна робота № 3.